When the variable in a comparison is not in the context, the condition gets treated as `False`. If `num_drummers` is not in the context in the preceding example, the comparison fails and the block does not get shown.

Conditional clauses can be combined using Boolean operators, like those in Python. You can use `and`, `or`, and `not` in an `{% if %}` tag. The same order of precedence as Python gets applied. If `and` and `or` are in the same clause, `and` gets higher precedence. The following code checks for the "truthiness" of `musicians` and compares `num_venues` to three:

```
{% if musicians and num_venues > 3 %}
    You have lots of choice!
{% endif %}
```

More complex conditional blocks can be constructed by adding `{% elif %}` and `{% else %}` tags to your conditional. These tags behave like their Python counterparts. As you might expect, the comparison clauses that work in `{% if %}` work in `{% elif %}` as well. The following checks for `num_musicians` being greater than 0 and less than 6, and then if that condition fails, it checks if `num_musicians` is greater than or equal to 6, and if that condition fails, you are told that you have zero musicians:

```
{% if num_musicians > 0 and num_musicians < 6 %}
    Band sized
{% elif num_musicians >= 6 %}
    Big-band sized
{% else %}
    Your band needs people
{% endif %}
```

3.5.2 Looping blocks

Consider a case in which you have a list of instruments that you want to output in an HTML bullet list. To do this cleanly, you need to loop over your data. The `{% for %}` tag is similar to a `for` statement in Python, allowing you to iterate over a container. Just like Python, the tag takes the name of a variable used inside the looping context and an object to iterate over. The value in the looping context gets used like any other variable, often displayed using double braces.

One use of the `{% for %}` tag is to construct `<ul>`, `<ol>`, or `<table>` structures in HTML. In all these cases the `{% for %}` tag goes inside the containing structure, with the contents of the `{% for %}` block being what you want to be repeated. The following code outputs a `<ul>` tag with its insides constructed by looping over the `instruments` value, rendering `<li>` tags:

```
<ul>
    {% for instrument in instruments %}
        <li> {{instrument.name}}: {{instrument.cost}}</li>
    {% endfor %}
</ul>
```